《现代C++编程实现》电子版：https://leanpub.com/c01

C++程序结构

YouTube频道： hwdong

博客： hwdong-net.github.io

B站: hw-dong

C++程序结构

- main()函数
- 语句
- 注释（多行或单行）
- 预处理指令(包含、宏定义)
- 函数（程序入口主函数main）
  - 变量和常量（数据）
  - 表达式语句（数据处理）
  - 控制语句、返回语句等
- 名字空间
- 标识符

最简单的C++ 程序

- 这是一个最简单的C++ 程序。程序执行后直接结束。

```
int main() {
    return 0;
}
```

main函数

- C++程序是由一些**函数**构成的，每个C++程序都执行唯一的叫作main的**主函数**。

```
int main() {
    return 0;
}
```

main函数

- int 表示 main 函数执行后会返回一个整数值。
- 一对花括号 {} 包围的部分是函数体，包含了程序要执行的语句。
- return 0; 是一条返回语句，它结束函数的执行，返回值0给**调用者**。对于main函数，这个值将返回给操作系统。

```
int main() {
    return 0;
}
```

函数

- 这个程序的核心就是 main 函数。那么，什么是函数呢?

```
int main() {
    return 0;
}
```

函数

- **函数**是一个具有特定功能的、可重复使用的代码块。它可以接收输入（称为**参数**），执行一系列操作，并可能返回一
- 个结果（称为**返回值**）。C++ 程序由一个或多个函数组成，其中必须有一个名为 main 的函数——它是程序的入口，
- 程序从这里开始执行。

```
int main() {
    return 0;
}
```

函数的定义格式

- **返回类型、函数名、参数列表、函数体**

```
<返回类型> <函数名>(<参数列表>) {
    // 函数体：包含多条语句
}
```

```
int main() {
    return 0;
}
```

函数的定义格式

- **函数体**：由一对花括号 {} 括起来的若干语句，描述了函数的具体功能。
- 函数体中有一条语句return 0; 是一条返回语句，它结束函数的执行，并返回值 0 。对于main函数，返回者被返回给操作系统。

```
int main() {
    return 0;
}
```

函数的定义格式

- **返回类型：**指定函数执行后返回的数据类型。例如 int 表示返回整数，double 表示返回浮点数， void 表示不返回任何值。

```
int main() {
    return 0;
}
```

函数的定义格式

- **函数名**：函数的名称，用于在其他地方调用它。

```
int main() {
    return 0;
}
```

函数的定义格式

- **参数列表**：函数可以接受的输入，写在圆括号内。多个参数用逗号分隔。如果没有参数，括号内可以为空（或写 void ）。

```
int main() {
    return 0;
}
```

整数求和函数

- 假设我们想编写一个函数，计算两个整数的和。可以这样定义：

```
int add(int x, int y) {    // 函数名为 add，接受两个 int 参数 x 和 y
    int sum = x + y;       // 计算和
    return sum;            // 返回结果
}
```

整数求和函数

- 假设我们想编写一个函数，计算两个整数的和。可以这样定义：

```
int add(int x, int y) {    // 函数名为 add，接受两个 int 参数 x 和 y
    int sum = x + y;       // 计算和
    return sum;            // 返回结果
}
```

- 返回类型是 `int`，表示函数会返回一个整数。
- 函数名是 `add`。
- 参数列表是 `(int x, int y)`，说明函数需要两个整数作为输入。
- 函数体内计算了 `x + y`，并将结果用 `return` 语句返回。

函数的调用

- 定义好函数后，我们可以在程序的其他地方（例如 main 函数中）**调用**它。调用函数时，需要写出函数名，并在括号内提供与参数列表匹配的实际值（称为**实参**）。

```
int main() {
    int a = 5;
    int b = 3;
    int result = add(a, b);    // 调用 add 函数，将 a 和 b 作为实参传递
    std::cout << "5 + 3 = " << result << std::endl;
    return 0;
}
```

```
#include <iostream>

// 定义 add 函数
int add(int x, int y) {
    int sum = x + y;
    return sum;
}

int main() {
    int a = 5;
    int b = 3;
    int result = add(a, b);    // 调用 add 函数，将 a 和 b 作为实参传递
    std::cout << "5 + 3 = " << result << std::endl;
    return 0;
}
```

1. main 函数开始执行，定义变量 a 和 b 。

2. 执行 add(a, b) 时，程序跳转到 add 函数，将 a 的值赋给 x ， b 的值赋给 y 。

3. 执行 add 函数体，计算 x + y 得到 8 ，并通过 return 将值返回给调用处。

4. 返回的值被赋给变量 result ，然后输出。

```
int main() {
    int a = 5;
    int b = 3;
    int result = add(a, b);    // 调用 add 函数，将 a 和 b 作为实参传递
    std::cout << "5 + 3 = " << result << std::endl;
    return 0;
}
```

无参数和无返回值的函数

- 如果函数不需要接收参数，参数列表可以为空；如果不需要返回结果，返回类型可以指定为 void 。

```
void sayHello() {           // 无参数，无返回值
    std::cout << "Hello!" << std::endl;
    // 没有 return 语句，或者可以写 return;
}
```

```cpp
#include <iostream>

void sayHello() {           // 无参数，无返回值
    std::cout << "Hello!" << std::endl;
    // 没有 return 语句，或者可以写 return;
}

int main() {
    sayHello();             // 调用函数
    return 0;
}
```

main 函数的特别之处

main 是一个特殊的函数，它是程序的入口点，由操作系统自动调用，我们不能在自己的代码中显式调用它。它的返回值通常用于向操作系统报告程序的执行状态：

- 返回 0 （或 EXIT_SUCCESS ）表示程序正常结束。
- 返回非零值（如 1 ）表示程序异常结束或出现错误。

```
int main() {
    // 程序代码...
    return 0;   // 正常结束
}
```

函数的作用

- **模块化**：将复杂任务分解为小的、独立的模块，每个函数完成一个特定功能。
- **代码复用**：避免重复编写相同代码，只需调用函数即可。
- **可读性**：通过函数名表达意图，使代码更容易理解。

语句

- C++程序的最小的完整执行指令都是以分号结尾的**语句**。

```
int main()
{
    return 0;
}
```

语句

- C++程序的最小的完整执行指令都是以分号结尾的**语句**。
- return语句结束函数执行，并可返回结果给调用者

```
int main()
{
    return 0;
}
```

注释

- 用以对程序代码进行说明，方便他人或今后阅读理解程序。

```
/*
    这是我的第一个C++程序
    This is my first C++ program
    作者:hwdong
    2020-02-24
*/

#include <iostream>    //输入输出流头文件
int main() { // main函数是程序的主入口，程序总是从这里执行
    return 0; //程序结束，返回结果0
}
```

/*和 */包围起来的多行注释

//开头的单行注释

- 多行注释不能嵌套

```
/*
    这是我的第一个C++程序
    This is my first C++ program
    作者:hwdong
    //修改日期：2020-02-24
*/

#include <iostream>    //输入输出流头文件
int main() { // main函数是程序的主入口，程序总是从这里执行
    return 0; //程序结束，返回结果0
}
```

块注释中不能包含/*或*/

但可以包含//

Hello World 程序

- 经典的 "Hello World" 程序，它将 "Hello World" 输出到控制台窗口：

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

输出"Hello World"并换行

执行此程序时，会在控制台窗口输出

```
Hello World
```

Hello World 程序

- 现代写法（C++23 标准，更简洁、更推荐）

```
#include <print>      // C++23 新增的头文件

int main() {
    std::println("Hello, Modern C++!");
    return 0;
}
```

执行后输出：

```
Hello, Modern C++!
```

关于 #include 的简单说明

- **#include<iostream>** 的作用是告诉预处理器：找到标准输入输出流库的头文件，并把它的内容插入到这里。
- 这样我们才能使用 std::cout 等输入输出功能。

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

关于 #include 的简单说明

- #include <print> 是为了使用 C++23 的 std::println

```
#include <print>      // C++23 新增的头文件

int main() {
    std::println("Hello, Modern C++!");
    return 0;
}
```

预处理指令

- 预处理指令是在编译器正式开始编译之前，由**预处理器**处理的特殊命令。它们**以 # 开头**，**结尾不需要分号**。
- 常见的预处理指令包括**文件包含**、**宏定义**和**条件编译**。

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

文件包含： #include

#include 指令用于将一个文件的内容插入到当前文件中。它有两种形式：

- **#include <文件名>** ： 用于包含标准库的头文件，预处理器会在系统指定的目录中查找。例如 #include<iostream> 。
- **#include "文件名"** ： 通常用于包含自己写的头文件，预处理器会先在当前目录查找，找不到再去系统目录。
- 无论是标准库还是自己写的头文件，使用 #include 后，我们就可以使用其中声明的函数、类或对象。

文件包含： #include

- 预处理器用文件内容替换掉该语句

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

#include <iostream>

预处理指令

在系统路径下寻找库文件

文件包含： #include

- 预处理器用文件内容替换掉该语句

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

#include “iostream”

预处理指令

先在当前目录，后在系统路径下寻找库文件

宏定义： #define

- #define 用于定义**宏**，它可以为一个常量或表达式起一个名字。例如：

```
#define PI 3.14159
```

- 这条指令告诉预处理器：在后面的代码中，只要出现 PI ，就把它替换成 3.14159 。这种替换发生在编译之前，因此称为**宏替换**。

宏定义： #define

- #define 用于定义**宏**，它可以为一个常量或表达式起一个名字。例如：

```
#include <iostream>
#define PI 3.14159

int main() {
    std::cout << PI * 2 * 2 << std::endl;
    return 0;
}
```

预处理器会将 PI 替换成 3.14159

宏定义可以提高可读性和可维护性。

条件编译： #ifdef 等

- 条件编译指令让预处理器根据条件决定哪些代码参与编译。常用的条件编译指令有 #ifdef 、#ifndef 、#endif 以及 #if 、#elif 、#else 。
- **示例：调试开关**

```
#include <iostream>
#define DEBUG  // 定义 DEBUG 宏

int main() {
#ifdef DEBUG
    std::cout << "正在调试模式中..." << std::endl;
#endif
    std::cout << "程序正常运行" << std::endl;
    return 0;
}
```

条件编译： #ifdef 等

- **示例：头文件保护**

在实际项目中，头文件可能被多个源文件包含，如果没有保护机制，会导致重复定义错误。使用条件编译可以轻松解决：

```
// myheader.h
#ifndef MYHEADER_H    // 如果 MYHEADER_H 未定义
#define MYHEADER_H    // 则定义它，并编译下面的内容

// 头文件的实际内容
void myFunction();

#endif                // 结束条件编译
```

```
// myheader.h
#ifndef MYHEADER_H    // 如果 MYHEADER_H 未定义
#define MYHEADER_H    // 则定义它，并编译下面的内容

// 头文件的实际内容
void myFunction();

#endif                // 结束条件编译
```

- 当预处理器第一次处理这个文件时， MYHEADER_H 未定义，因此会执行 #define 并包含函数声明。如果之后再次遇到 #include "myheader.h"，此时 MYHEADER_H 已定义， #ifndef 条件不成立，整个头文件内容被跳过，从而避免了重复定义。

条件编译指令

- #if、#else、#elif 、#endif

```
#if 0
main(){

}
#else
int main(){
  return 0;
}
#endif
```

条件编译指令

- #ifdef、#else、#elif 、#endif

```
#ifdef UNIX
...
#endif

int main(){
    /*... */
}
```

标准输入输出库、 cout和输出流运算符<<

标准输入输出库

- 需要与用户进行交互，比如在屏幕上显示信息或从键盘读取输入。C++ 标准库提供了 **输入输出流库**（iostream）来实现这些功能。

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

标准输入输出库

- 要使用输入输出功能，需要在程序开头包含头文件 <iostream>：

```
#include <iostream>
```

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

标准输出对象 cout

- cout 是标准输出流对象，代表终端窗口（或控制台）。
- std 是一个**名字空间（namespace）**，它把标准库中的所有名字（如 cout、cin、endl 等）都组织在一起，避免和其他名字冲突。
- std::cout 就是“标准名字空间里的 cout”。

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

输出运算符 <<

- 输出运算符 << 用于将数据发送给 cout ，从而显示在屏幕上。

```
std::cout << "hello world";
```

- 这里 << 运算符将字符串 "hello world" 插入到 cout 中，然后显示在屏幕上。

换行符：std::endl

- 使用 std::endl （它会换行并强制刷新输出缓冲区，让内容立即显示）：

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

换行符

连续输出

- << 运算符可以连续使用，因为 cout << ... 的结果仍然是 cout，所以可以接着写：

```
std::cout << "Hello World" << std::endl;
```

等价于：

```
std::cout << "Hello world";
std::cout << std::endl;
```

运算符与操作数

- 在程序中，我们经常需要对**数值**进行计算，比如加减乘除、求余等。这些计算是通过**运算符**来完成的。
- **运算符** 是告诉计算机“对哪些数做什么操作”的符号。参与运算的数称为**操作数**。

```
// 计算周长：2 × (长 + 宽)
std::cout << "周长 = " << 2 * (5.8 + 3.4) << std::endl;

// 计算面积：长 × 宽
std::cout << "面积 = " << 5.8 * 3.4 << std::endl;
```

```
#include <iostream>

int main() {
    // 直接使用数值（常量）进行计算
    std::cout << "矩形的长 = 5.8，宽 = 3.4" << std::endl;

    // 计算周长：2 × (长 + 宽)
    std::cout << "周长 = " << 2 * (5.8 + 3.4) << std::endl;

    // 计算面积：长 × 宽
    std::cout << "面积 = " << 5.8 * 3.4 << std::endl;

    // 其他运算示例
    std::cout << "5.8 + 3.4 = " << 5.8 + 3.4 << std::endl;
    std::cout << "5.8 - 3.4 = " << 5.8 - 3.4 << std::endl;
    std::cout << "5.8 / 3.4 = " << 5.8 / 3.4 << std::endl;

    // 整数求余示例（% 只适用于整数）
    std::cout << "10 ÷ 3 的余数 = " << 10 % 3 << std::endl;

    return 0;
}
```

变量：命名的内存块

- 直接把数字写死在程序里（称为“硬编码”）

```
std::cout << 5.8 * 3.4 << std::endl;    // 矩形面积
std::cout << 3.14159 * 2.5 * 2.5 << std::endl;    // 圆面积
```

变量：命名的内存块

- **变量**本质上是一块带有名字的内存区域。程序可以通过这个名字来访问或修改内存里的数据，就像给一个抽屉贴上标签（变量名），往里面放东西（数据），需要时再通过标签取出来。

```
#include <iostream>

int main() {
    double r;            // 定义一个 double 类型的变量，
    r = 2.5;             // 把数值 2.5 存入变量 r
    double area = 3.14159 * r * r;  // 用 r 的值计算
    std::cout << "面积为: " << area << std::endl;
    return 0;
}
```

数据类型

- 每个变量都必须有一个**数据类型**，它决定了变量能存储什么种类的数据以及占用多少内存。C++ 提供了多种内置数据类型，常用的有：

- `int`：整数类型，用于存储整数（如 10、-3、0）
- `double`：双精度浮点类型，用于存储带小数点的实数（如 3.14、-0.5）
- `char`：字符类型，用于存储单个字符（如 'A'、'#'）

变量的定义和初始化

- 定义变量的基本语法是： 数据类型 变量名;

```
int i;          // 定义一个名为 i 的 int 型变量
double r;       // 定义一个名为 r 的 double 型变量
char ch;        // 定义一个名为 ch 的 char 型变量
```

变量的定义和初始化

- 为了避免使用未初始化的变量导致不可预测的结果，建议在定义时同时进行**初始化**，即赋予一个初始值。

```
int i = 10;            // 使用 = 初始化
int j{20};             // 使用 {} 初始化（C++11 开始推荐）
double r{3.14};        // 浮点数初始化
char ch{'A'};          // 字符初始化
```

变量的赋值

除了初始化，我们还可以在程序执行过程中修改变量的值，这称为**赋值**。赋值使用 = 运算符：

```
int x;          // 定义变量（未初始化）
x = 5;          // 赋值，将 5 存入变量 x
x = x + 2;      // 先计算 x+2 得到 7，再赋值给 x，现在 x 的值为 7
```

变量的命名规则

- 变量名只能由字母、数字、下划线组成。
- 不能以数字开头。
- 不能是 C++ 的关键字（如 `int`、`return`、`double` 等）。
- 建议使用有意义的英文单词或拼音，如 `age`、`score`、`radius`。

圆面积程序，理解每一行的作用：

```
#include <iostream>

int main() {
    double r;                          // 定义变量 r（未初始化）
    r = 2.5;                           // 赋值，存储半径
    double area = 3.14159 * r * r; // 定义并初始化 area，存储计算结果
    std::cout << "面积为: " << area << std::endl;
    return 0;
}
```

名字空间

- 为什么要在std::cout加上 std:: 这个前缀呢？这其实和 C++ 中的**名字空间**有关。名字空间的出现，是为了解决程序中的名字冲突问题。
- **名字冲突**：不同的库可能出现同名，如同生活中出现同名。
- 机械学院有个学生叫“张伟”，计算机学院也有个学生叫“张伟”。
- 用**所属学院限定**解决冲突: “机械学院的张伟”、“计算机学院的张伟”。
- C++ 引入了**名字空间（namespace）**解决了名字冲突！

名字空间

- 标准库中的所有名字（如 cout 、cin 、string 等）都放在名为 std 的名字空间中。因此，要使用标准库中的名字，通常需要加上 std:: 前缀，就像“计算机学院的张伟”一样明确。
- std 是 "standard"（标准）的缩写，表示这是C++标准库提供的名字空间。

使用名字空间的三种方式

1. 完全限定（最推荐）

- 每次使用时都显式地加上 std:: 前缀。这种方式最清晰、最安全，能明确指定名字的来源。

```
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

使用名字空间的三种方式

2. 使用 using 声明

- 如果觉得每次写 std:: 太繁琐，可以用 using 声明引入需要的特定名字。之后就可以直接使用，不用加前缀。

```
#include <iostream>
using std::cout;     // 只引入 cout
using std::endl;     // 只引入 endl

int main() {
    cout << "Hello, world!" << endl;    // 直接使用
    return 0;
}
```

使用名字空间的三种方式

3. 使用 using 指令（不推荐）

- using namespace std; 会将 std 名字空间中的所有名字一次性引入当前作用域。虽然写起来最方便，但风险也最大：如果当前作用域中已有同名实体，或者未来标准库新增了与你代码中同名的名字，就会导致意想不到的名字冲突。

```
#include <iostream>
using namespace std;    // 将 std 中的所有名字都引入

int main() {
    cout << "Hello, world!" << endl;    // 可以直接使用
    return 0;
}
```

现代 C++ 建议

- **优先使用完全限定方式**（std::cout），代码意图明确，可读性和可维护性最好。
- 如果觉得写 std:: 过于繁琐，可以在函数内部或小范围内使用 using 声明，引入少数几个常用名字。
- 避免在头文件中使用 using namespace std;，以免影响包含该头文件的所有源文件。

扩展：定义自己的名字空间（可选）

- 我们也可以使用 namespace 关键字定义自己的名字空间，将相关的函数、变量、类等组织在一起，避免与他人代码中的同名实体发生冲突。

定义名字空间的基本语法

```
namespace 名字空间名 {
    // 在这里定义函数、变量、类等
}
```

扩展：定义自己的名字空间（可选）

- 假设我们编写了一些数学运算函数，可以把它们放在一个名为 MyMath 的名字空间中：

```
// 定义自己的名字空间 MyMath
namespace MyMath {
    int add(int x, int y) {
        return x + y;
    }

    int subtract(int x, int y) {
        return x - y;
    }
}
```

```cpp
#include <iostream>

// 定义自己的名字空间 MyMath
namespace MyMath {
    int add(int x, int y) {
        return x + y;
    }

    int subtract(int x, int y) {
        return x - y;
    }
}
```

```cpp
int main() {
    // 使用 MyMath 中的函数时，需要加上 MyMath:: 前缀
    int sum = MyMath::add(5, 3);
    int diff = MyMath::subtract(5, 3);

    std::cout << "5 + 3 = " << sum << std::endl;
    std::cout << "5 - 3 = " << diff << std::endl;

    return 0;
}
```

嵌套名字空间

- 名字空间可以嵌套定义，例如：

```
namespace Company {
    namespace DepartmentA {
        void func() { ... }
    }
    namespace DepartmentB {
        void func() { ... }
    }
}
```

字符串和字符

- 我们经常需要处理文字信息，比如输出提示语、存储用户名等。这些文字可以是单个字符，也可以是一串字符。

字符（Character）

- 用**单引号** ' 括起来的一个字符，表示一个单独的字符。例如 'A' 、'3' 、'#' 、'@' 、'*' 等。

```
int main() {
    std::cout << 'A' << std::endl;          // 输出大写字母 A
    std::cout << '3' << std::endl;          // 输出数字字符 3
    std::cout << '#' << std::endl;          // 输出符号 #
    std::cout << '@' << std::endl;          // 输出 @ 符号
    std::cout << '*' << std::endl;          // 输出星号 *
    // 输出空格字符
    std::cout << "这里有一个空格字符: " << ' ' << "后面是结束" << std::endl;
    return 0;
}
```

字符串（String）

- 用**双引号** " 括起来的一系列字符，表示一个字符串。例如：

```
std::cout << "Hello World" << std::endl;
std::cout << "欢迎学习现代 C++" << std::endl;
```

转义字符

- 有些字符在字符串中有特殊含义，比如双引号 " 本身、换行符等。要表示这些字符，需要使用反斜杠 \ 进行**转义**。
- 常见的转义字符有：
    - `\n`：换行符
    - `\t`：制表符（用于对齐）
    - `\\`：反斜杠本身
    - `\"`：双引号
    - `\'`：单引号

```
#include <iostream>

int main() {
    std::cout << "姓名\t年龄\t成绩" << std::endl;        // 使用制表符对齐
    std::cout << "路径：C:\\Windows\\System32" << std::endl; // 输出反斜杠
    std::cout << "他说：\"你好！\"" << std::endl;          // 输出双引号
    return 0;
}
```

输出：

```
姓名    年龄    成绩
路径：C:\Windows\System32
他说："你好！"
```

利用 cout 输出简单图案

- 通过组合字符和转义字符，可以在控制台输出有趣的图案。例如：

```
#include <iostream>

int main() {
    std::cout << "        *" << '\n';
    std::cout << "      *   *" << '\n';
    std::cout << "    *   *   *" << '\n';
    std::cout << "  *   *   *   *" << '\n';
    return 0;
}
```

常量定义：constexpr 现代方式

- **传统方式回顾：** #define **宏定义**

```
#define PI 3.14159
```

- 预处理器会在编译前将代码中所有 PI 替换为 3.14159 。这种方式虽然简单，但存在一些缺点（如无类型检查、作用域混乱、调试困难等），因此现代 C++ 不推荐在新代码中使用。

现代方式： constexpr 常量

- 从 C++11 开始，标准引入了 **constexpr** 关键字，用于定义编译期常量。它具有类型安全、作用域可控、可调试等优点，是定义常量的推荐方式。

基本语法：

```
constexpr 类型 常量名 = 值;
```

基本语法:

```
constexpr 类型 常量名 = 值;
```

例如:

```
constexpr double PI = 3.141592653589793;
constexpr int MAX_SIZE = 100;
constexpr char GRADE = 'A';
```

```
constexpr double PI = 3.141592653589793;
constexpr int MAX_SIZE = 100;
constexpr char GRADE = 'A';
```

- `constexpr` 表示该值在编译时就能确定，且不会改变。
- 常量有明确的类型，编译器会进行类型检查。
- 常量遵循作用域规则，可以定义在函数内、类内或命名空间内，不会污染全局。

```cpp
#include <iostream>

int main() {
    constexpr double PI = 3.14159;   // 定义常量
    double r = 2.5;
    double area = PI * r * r;
    std::cout << "面积为: " << area << std::endl;
    return 0;
}
```

为什么推荐 constexpr？

- 因此，**编写新代码时应优先使用** constexpr 定义常量。#define 只需在阅读旧代码或某些特殊场景（如条件编译）时了解即可。

方面	`#define` 宏定义	`constexpr` 常量
类型安全	无（纯文本替换）	有（明确类型）
作用域	全局有效（从定义点到文件结束）	遵循作用域规则（可局部定义）
调试友好	替换后看不到原名	调试器能识别常量名
编译期计算	是（预处理阶段）	是（编译阶段，支持更复杂计算）

标准输入流对象 cin

- 为了让程序能处理用户实时输入的数据，我们需要使用标准输入流对象 cin 。
- cin 是 C++ 标准库中定义的输入流对象，代表键盘输入。它和 cout 一样，属于 std 名字空间。使用 cin 需要包含 <iostream> 头文件。

基本用法： cin >> 变量

- 输入运算符 >> 用于从 cin 读取数据并存入指定的变量。例如：

```
int age;
std::cin >> age;    // 等待用户输入一个整数，存入变量 age
```

- 程序执行到 std::cin >> age; 时会暂停，等待用户输入数据并按回车。输入的数据会被转换为变量的类型（此处为 int），然后存入变量。

示例：计算圆的面积（用户输入半径）

```
#include <iostream>

int main() {
    constexpr double PI = 3.14159;
    double radius;

    std::cout << "请输入圆的半径：";
    std::cin >> radius;   // 读取用户输入的半径

    double area = PI * radius * radius;
    std::cout << "圆的面积为：" << area << std::endl;

    return 0;
}
```

示例：计算圆的面积（用户输入半径）

```
#include <iostream>

int main() {
    constexpr double PI = 3.14159;
    double radius;

    std::cout << "请输入圆的半径：";
    std::cin >> radius;   // 读取用户输入的半径

    double area = PI * radius * radius;
    std::cout << "圆的面积为：" << area << std::endl;

    return 0;
}
```

```
请输入圆的半径：2.5
圆的面积为：19.6349
```

输入多个值

- 可以连续使用 >> 读取多个数据，数据之间用空格或换行分隔：

```
int a, b;
std::cin >> a >> b;    // 依次读取两个整数
```

注意事项

- 输入的数据类型必须与变量类型匹配。如果输入类型不匹配（例如输入字母给 int 变量），会导致输入失败，变量值可能保持未定义状态，后续计算结果出错。关于输入错误处理，将在后续章节学习。
- cin 读取字符串时，默认以空白字符（空格、制表符、换行）作为分隔符。例如输入 "Hello World" 只会读取 "Hello"，如果需要读取包含空格的整行，可以使用 std::getline 。

标准库类型 std::string

- 除了内置的数据类型（如 int 、double 、char ），C++ 标准库还提供了许多实用的类型，其中最常用的就是std::string，用于处理字符串。
- 要使用 std::string，需要包含头文件 <string> 。

定义和初始化

```
#include <string>

std::string s1;                // 空字符串
std::string s2 = "hello";       // 拷贝初始化
std::string s3("world");        // 直接初始化
std::string s4(5, '*');         // 包含 5 个 '*' 的字符串： "*****"
```

基本操作

- **输入**：可以用 `cin >>` 读取一个单词（遇空白停止）。

```
std::string name;
std::cin >> name;
```

- **输出**：直接用 `cout <<` 输出。

```
std::cout << s2 << std::endl;
```

- **拼接**：使用 `+` 运算符连接字符串。

```
std::string full = s2 + " " + s3;   // "hello world"
```

- **获取长度**：调用 `size()` 成员函数。

```
std::cout << full.size() << std::endl;  // 输出 11
```

- **访问字符**：像数组一样用下标访问。

```
char first = full[0];   // 'h'
```

示例：读取姓名并问候

```
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cout << "请输入你的名字：";
    std::cin >> name;
    std::cout << "你好，" << name << "！" << std::endl;
    return 0;
}
```

常用成员函数

- `s.substr(pos, count)`：返回从 `pos` 开始、长度为 `count` 的子串。
- `s.find(str)`：查找子串 `str` 第一次出现的位置，返回下标（若未找到返回 `std::string::npos`）。
- `s.empty()`：判断字符串是否为空。
- `s.clear()`：清空字符串。

```
std::string s = "hello world";
std::string sub = s.substr(0, 5);   // "hello"
size_t pos = s.find("world");        // 返回 6
```

程序控制流简介

- 到目前为止，我们编写的程序都是顺序执行的——从main 的第一条语句开始，一条一条执行到最后。
- 但实际程序往往需要根据条件做出选择，或者重复执行某段代码。C++提供了**控制语句**来实现这些功能。

if条件语句

- if根据条件决定是否输出“及格”

条件判断示例：

```
int score = 85;
if (score >= 60) {
    std::cout << "及格" << std::endl;
}
```

while循环语句

- while让输出重复3次。

循环重复示例：

```
int count = 1;
while (count <= 3) {
    std::cout << "第" << count << "次循环" << std::endl;
    count = count + 1;
}
```

关注我

博客： hwdong-net.github.io

Youtube频道

hwdong

@hwdong · 5.01K subscribers · 558 videos

博客：https://hwdong-net.github.io >

youtube.com/c/4kRealSound and 4 more links